

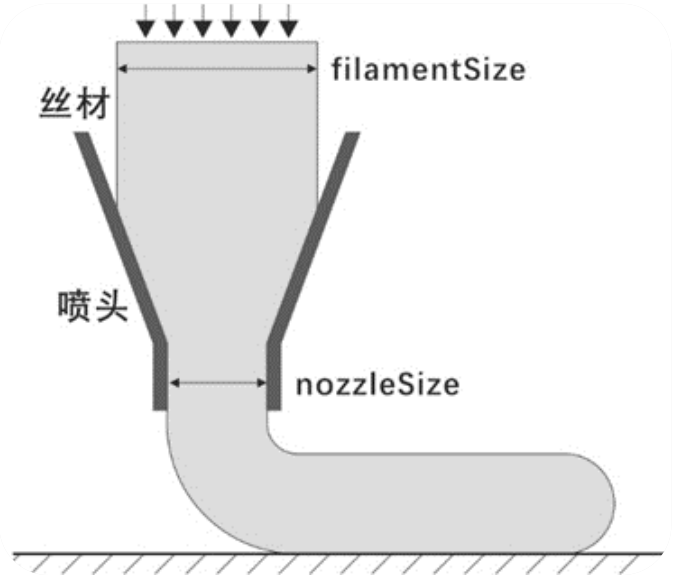
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

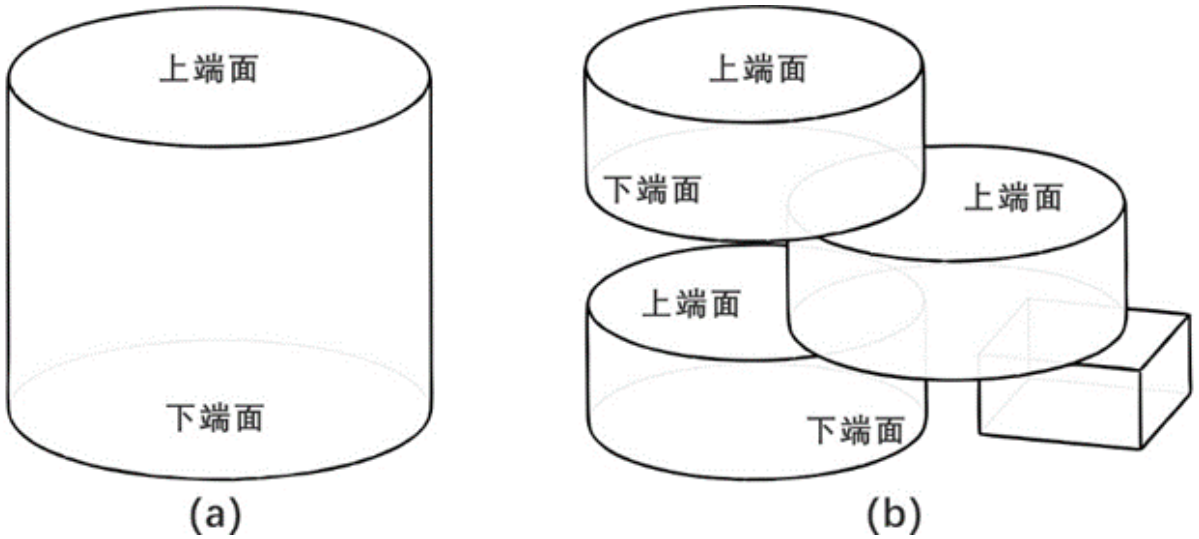
第11章 端面封闭与代码生成

- 端面识别与切片分区
- 打印代码生成



1. 端面识别与切片分区

- 三维打印分层叠加制造的工艺特点决定了在垂直于模型生长方向的平面上，将出现一系列**端面**，其中最为典型的就是模型的**上端面**和**下端面**。
  - 以圆柱体模型为例，如图，如果圆柱轴线平行于坐标系Z轴，那么圆柱的底面就是端面，其中下底面就是下端面，上底面就是上端面。
- 在三维打印中，需要**对端面**及其附近进行**密实填充**，以封闭模型。
- 从填充路径生成的角度来说，对端面切片，必须设置**较小的路径间距**（事实上恰为喷头直径），以保证填充后路径之间**无间隙**。
  - 考虑到已在第八、九章对填充路径的生成方法做过详细介绍，因此只要合理设置填充参数，对端面进行密实填充是容易实现的。
- **问题：如何识别模型上的端面，特别是对于那些较为复杂的模型？**



三维模型上的上端面和下端面

两种思路

从模型出发 V.S. 从切片出发

1. 端面识别与切片分区

- **1.1. 端面密实填充区域提取：下端面识别为例**

- 对任意封闭轮廓 c_1、c_2，分别位于相邻的第 i、i+1 层切片上，不难想象，图中阴影区域就是暴露在模型外需要被封闭填充的下端面。

- 记阴影区域轮廓为 d，根据平面区域的布尔运算原理，只需对轮廓 c_2 和 c_1 做布尔差运算，即让 c_2 减去 c_1，就可得到轮廓 d，即

$$d = c_2 - c_1. \qquad (11\text{-}1)$$

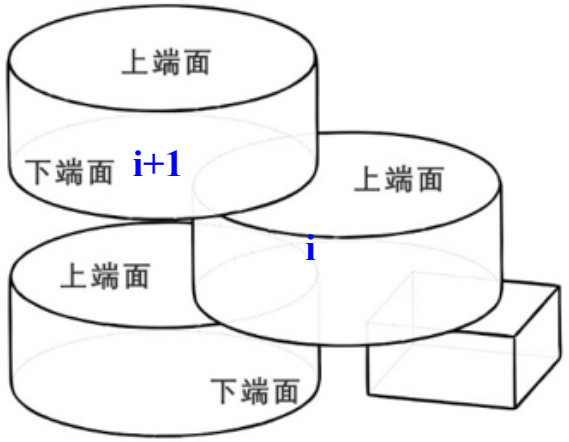
- 进一步地，对轮廓 c_1 和 c_2 求交，或者让轮廓 c_2 减去 d，就可求得 c_1、c_2 位于模型内部的重叠部分，即 d 关于 c_2 的互补轮廓，记这部分轮廓为 $\bar{d}$。

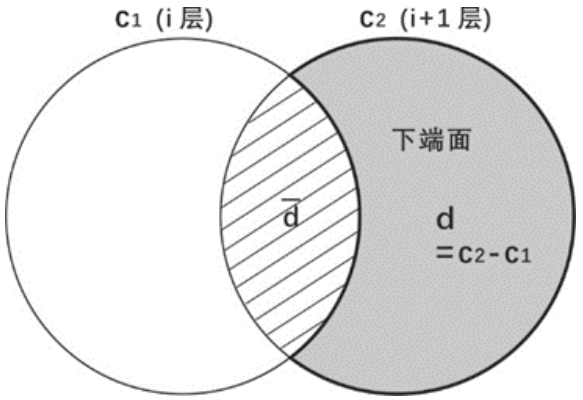
小结

通过布尔运算，将上层切片轮廓 c_2 围成的区域分成了两个部分，一个为 d，一个为 $\bar{d}$。

✓ d 就是模型下端面，必须进行**密实填充**；

✓ 对 $\bar{d}$，为了节省材料，节约打印时间，可进行**稀疏填充**。

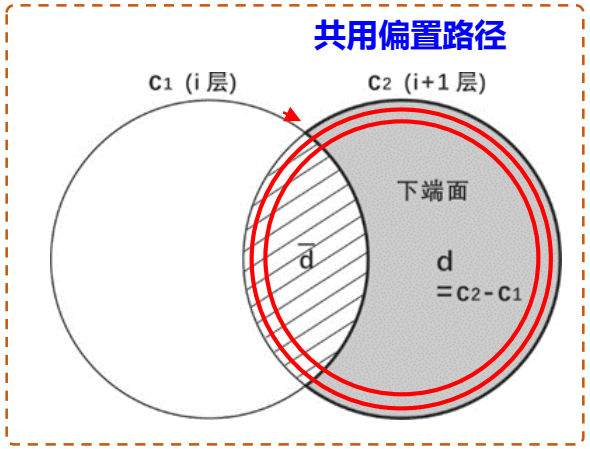




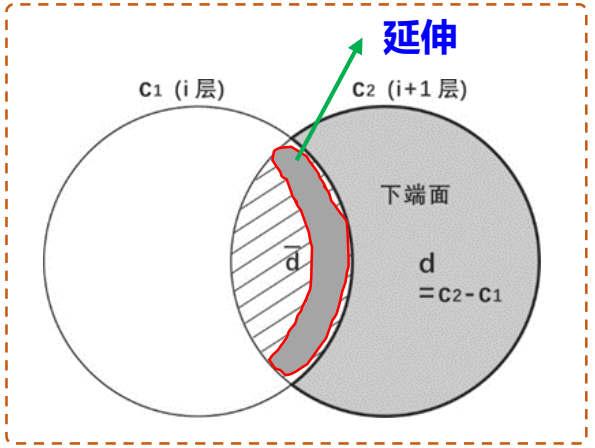
相邻切片层密实填充区域裁剪识别方法

1. 端面识别与切片分区

- **1.1. 端面密实填充区域提取：下端面识别为例**



以上分区方法忽略的两个重要因素



路径填充形式

- 为了兼顾精度和效率，会在打印区域外围使用轮廓填充路径，而在区域内部使用平行填充路径。
- d、$\bar{d}$ 同属 c_2，为避免在工件表面出现轮廓拼接印记，在生成路径时，它们应该**共用 c_2 的偏置曲线作为轮廓填充路径。**

密实填充强度

- 没有为d预设过渡区域，恰在d、$\bar{d}$边界进行疏密填充切换不理想，只会降低模型下端面强度，削弱其对上方材料的支撑作用。
- 较为理想的**密实填充边界应向$\bar{d}$区域延伸一定的距离。**

1. 端面识别与切片分区

- **1.1. 端面密实填充区域提取：对d、$\bar{d}$两个区域进行修正的流程**

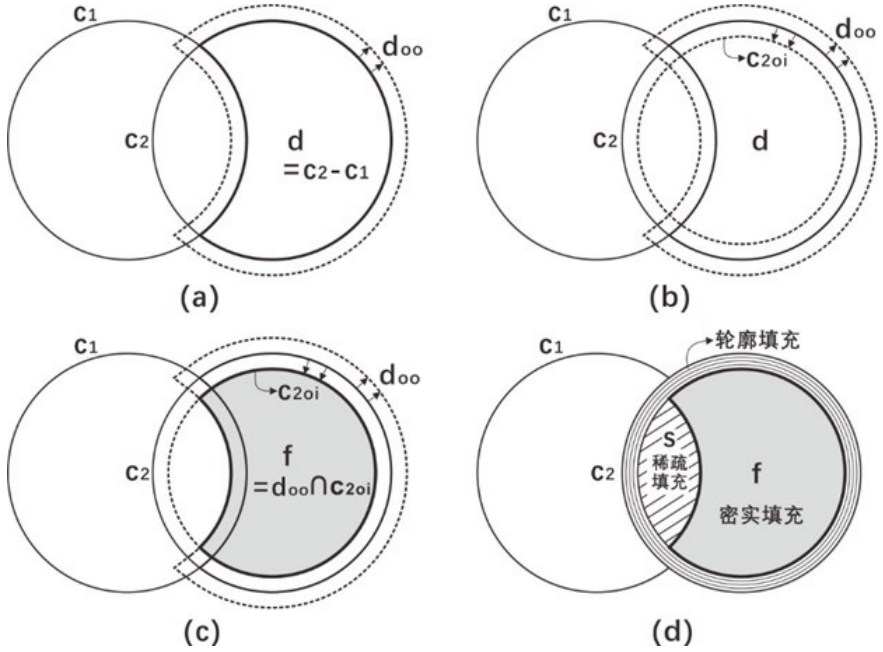
(1) 如图a所示，对 d 区域向外延伸，求取轮廓 d 的向外偏置曲线 d_{oo}，偏置距离为 x_o。

- 区域 d 向 $\bar{d}$ 延伸了一段距离，这有利于提升下端面的支撑强度；但同时，d_{oo} 在不该延伸的 c_1、c_2 轮廓外侧也得到了延伸，这是不希望看到的。

(2) 如图b所示，对 c_2 进行向内偏置，偏置距离为 x_i，得到偏置曲线 c_{2oi}（offset inside）。

(3) 如图c所示，对 d_{oo} 和 c_{2oi} 进行求交操作，得到最终密实填充区域，记其为f（full fill），即：$f = d_{oo} \cap c_{2oi}.$ (11-2)

(4) 如图d所示，将偏置轮廓 c_{2oi} 减去 f，可得到稀疏填充区域，记其为s（sparse fill），则：$s = c_{2oi} - f.$ (11-3)



下端面密实填充区提取流程

需要说明的是：

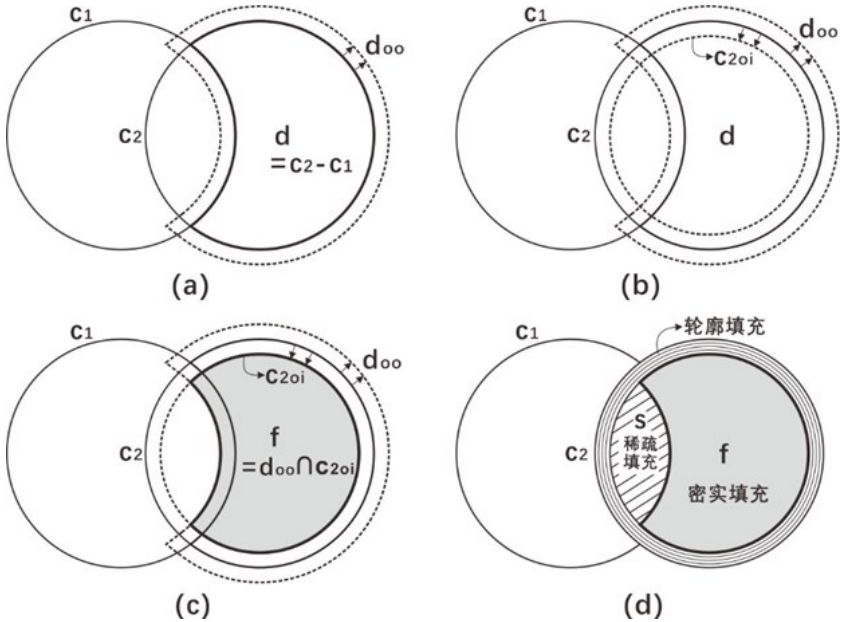
以上得到的轮廓 f、s 均在轮廓 c_2 所在的平面上。

1. 端面识别与切片分区

- **1.1. 端面密实填充区域提取：对d、$\bar{d}$两个区域进行修正流程**

口 几点说明

- 在以上讨论中，从轮廓 c_2 到 c_{2oi} 的偏置距离为 x_i，x_i 就是第八章中讨论过的轮廓路径填充带宽（参见代码8-7），它表示打印后模型的外壳厚度 shellThk。此外，以上暂记从轮廓 d 到 d_{oo} 的偏置距离为 x_o，x_o 值表示了密实填充的端面向模型内部的延伸或重叠量，简单起见，在这里，不妨直接对 x_o 赋 shellThk 值。**综上所述，在数值上，以上：$x_o = x_i$ = shellThk。**
- 在以上推导过程中，为了方便说明和理解，默认轮廓 c_1、c_2、d、f、s 等均为单一曲线；但事实上，**每个符号都可以表示一组轮廓**，如c_1、c_2可以分别表示指定切片层上的若干轮廓，f、s可以表示若干密实填充区域轮廓和若干稀疏填充区域轮廓。



下端面密实填充区提取流程

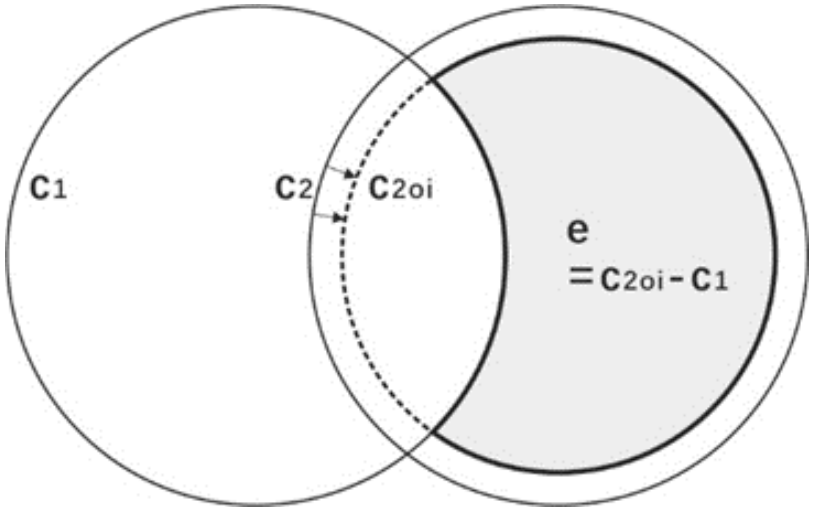
1. 端面识别与切片分区

- **1.1. 端面密实填充区域提取：如何判断是否有密实填充的下端面？**
- **以上讨论默认了下端面存在**，即：c_1、c_2 所表示的上下层轮廓之间**存在一定的偏差（d），**这种偏差在 c_2 区域内表现为模型下端面。
- 但实际情况可能是 c_1、c_2 形状大小完全一致，或它们之间存在偏差，但**偏差很小**，此时，在 c_2 区域内是否仍存在下端面？更具体地，**如何根据偏差的大小判断切片上是否存在下端面？**

- 如果存在下端面，那么整个下端面 d 需要被密实填充，而对 d 区域外围，使用轮廓路径进行填充，填充带宽为 **shellThk**。注意，**轮廓填充本身就是一种密实填充方式**！因此，如图所示，记

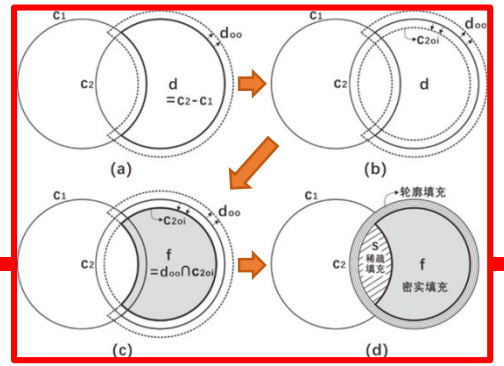
$$e = c_{2oi} - c_1, \qquad (11\text{-}4)$$

  - ✓ 如果 e 不为空，则 c_2 所在的切片存在下端面，此时需要进一步通过式(11-2)和(11-3)计算该切片的密实填充区 f 和稀疏填充区 s；
  - ✓ 如果 e 为空，则 c_2 所在的切片不存在下端面，此时由 c_{2oi} 围成的区域全为稀疏填充区。



下端面密实填充区提取流程

1. 端面识别与切片分区



• 1.1. 端面密实填充区域提取：代码实现

- 针对相邻**两切片层轮廓 c_1、c_2**，借助偏置运算和布尔运算，**可提取切片上的下端面密实填充区**，下面给出相应函数，函数名为 **pickFfRegions**，模块名为 **IdEndLayers**，具体实现如下：

IdEndLayers.py	端面切片识别模块，定义端面密实填充区域提取函数	11-1

```
1   def pickFfRegions(layer1, layer2, shellThk):          # 定义提取密实填充区函数
2       c1, c2 = layer1.contours, layer2.contours          # 获取c1、c2轮廓
3       ca = ClipperAdaptor()                              # 定义裁剪适配器对象ca
4       c2oi = ca.offset(c2, -shellThk)                    # 计算c2oi
5       if len(c2oi) == 0: return False                    # 如果c2oi为空，返回False
6       layer2.shellContours = c2oi                        # 对切片外壳内轮廓赋值
7       e = ca.clip(c2oi, c1, CT_DIFFERENCE) if len(c1) > 0 else c2oi        # 计算e，根据式(11-4)
8       if len(e) == 0: return False                       # 如果e为空，返回False
9       d = ca.clip(c2, c1, CT_DIFFERENCE) if len(c1) > 0 else c2    # 计算d，根据式(11-1)
10      doo = ca.offset(d, shellThk)                       # 计算doo
11      f = ca.clip(doo, c2oi, CT_INTERSECTION, c2[0].point(0).z)  # 计算f，根据式(11-2)
12      layer2.ffContours = f                              # 对切片密实填充轮廓赋值
13      return True                                        # 返回True
```

如果c_1为空，则$d = c_2$，$e = c_{2oi}$，这是因为Clipper库无法接受空集作为参数

- **输入参数**
  - layer1 ——下层切片
  - layer2 ——上层切片，layer2位于layer1上方
  - shellThk ——轮廓路径填充带宽，或外壳厚度
- **返回参数**
  - True，存在密实填充区；False，不存在密实填充区。

说明：

- 以上代码的前提条件是轮廓 **c_2 位于 c_1 的上方**，而且求取的是 c_2 轮廓上的下端面密实填充区。
- 事实上，只要在调用函数时**对 $layer_1$ 和 $layer_2$ 的位置进行对调**，就能获取 **c_1 轮廓上上端面**的密实填充区。
- 因此，**无需重复实现上端面密实区域提取函数**。

1. 端面识别与切片分区

- **1.1. 端面密实填充区域提取：代码实现**

- 在 **pickFfRegions** 函数中，得到的密实填充区 f 被保存至切片layer2 的 ffContours 中，同时将轮廓填充的内边界 c_{2oi} 保存至 layer2 的 shellContours 中，留作后用。为此，对Layer类进行了扩充，在其初始化函数中增加了 shellContours、ffContours 以及 sfContours 三个属性，其中 sfContours 用于保存切片上的稀疏填充区域边界，” sf “为 sparse fill 的缩写。

Layer.py　切片类，增加shellContours、ffContours、sfContours属性　11-2

```
class Layer:                                    # Layer类
    def __init__(self, z):
        ……                                      # 忽略其他属性
        self. shellContours = []                #        轮廓填充内边界轮廓
        self.ffContours= []                     # 密实填充区域轮廓
        self.sfContours = []                    # 稀疏填充区域轮廓
```

- **pickFfRegions**函数涉及一系列平面轮廓的布尔运算，为了简化布尔操作流程，在ClipperAdaptor类中添加了一个**clip**函数，具体实现如下：

ClipperAdaptor.py　Clipper库适配器类，添加clip函数，简化轮廓布尔计算流程　11-3

```
……                                              # 导入相关库
class ClipperAdaptor:                           # Clipper适配器类
    ……                                          # 忽略类中其他代码
    def clip(self, subjPolys, clipPolys, clipType, z = 0, minArea = 0.01):    # 定义clip函数
        clipper = Pyclipper()                   # 定义裁剪对象
        clipper.AddPaths(self.toPaths(subjPolys), PT_SUBJECT)  # 添加subject轮廓序列
        clipper.AddPaths(self.toPaths(clipPolys), PT_CLIP)     # 添加clip轮廓序列
        sln = clipper.Execute(clipType)         # 执行布尔运算
        slnPolys = self.toPolys(sln, z)         # 曲线类型转换
        for i in range(len(slnPolys) - 1, -1, -1):  # 删除面积过小封闭轮廓
            if math.fabs(slnPolys[i].getArea()) < minArea:
                del slnPolys[i]
        return slnPolys                         # 返回布尔运算结果
```

minArea用于筛选布尔运算得到的轮廓，如果轮廓面积小于minArea，则从结果中删除该轮廓，默认情况下，minArea等于0.01mm²

1. 端面识别与切片分区

- **1.2. 切片疏密填充区域分区**

注意：pickFfRegions 函数输入为**上下相邻的两层切片**，下一步需要对**所有切片**提取疏密分区。

直观思路

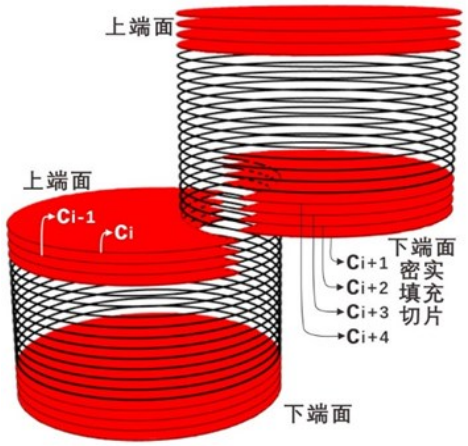
从下往上（或从上往下）**两两遍历**切片序列中的相邻切片，提取每层切片上的**密实填充区域**。

进一步，通过**布尔运算**还能获得每层切片上的**稀疏填充区域**。

事情真的如此简单？

真实多层切片案例分析

- 如图，以下端面密实填充区提取为例，对相邻层切片轮廓 c_i、c_{i+1}，可识别到 c_{i+1} 上的下端面，同时提取其密实填充区。
- 当检测切片层上移后，对 c_{i+1}、c_{i+2}，它们在XY平面上的投影完全重合，因此在 c_{i+2} 上不存在下端面，也不存在密实填充区。
- 在检测切片层对继续上移过程中，在 c_i、c_{i+1} 切片层附近，**始终只有一层切片（即c_{i+1}）为密实填充的下端面**，承担来自端面上方的所有材料重量，这显然是有问题的。



端面附近的多层密实填充切片

1. 端面识别与切片分区

- **1.2. 切片疏密填充区域分区**

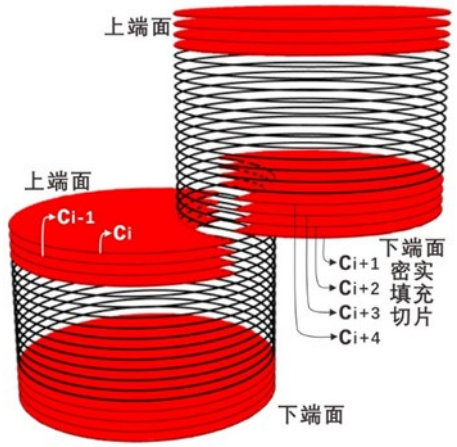
合理思路：是对 c_{i+1} 及其**上方若干层**的外延部分区域进行密实填充，以增加端面强度。

- 记需要密实填充的端面层数为 n，n 可由下式计算得到：

$$n = \mathrm{int}(H/h) + 1, \qquad (11\text{-}5)$$

式中 H 为端面厚度，h 为切片厚度，int 为取整符号。

- 基于此，在遍历过程中，对当前相邻层切片轮廓 c_i、c_{i+1}，**如果检测到 c_{i+1} 上存在下端面，那么有必要对 c_i和c_{i+2}、c_i和c_{i+3}、…、c_i和c_{i+n}切片层对展开进一步测试**，以找出 c_{i+1}、c_{i+2}、…、c_{i+n} 等 n 个切片层上的下端面密实填充区域。



端面附近的多层密实填充切片

1. 端面识别与切片分区

• 1.2. 切片疏密填充区域分区

根据以上讨论，下面直接给出针对一个三维模型**所有切片层**的**下端面密实填充区域分区函数**。函数名为**splitFfRegions**，写在**IdEndLayers**模块中，具体实现如下：

IdEndLayers.py　　定义所有切片层下端面密实填充区域寻找函数　　11-4

```
1   ……                                                  # 导入相关库，忽略其他代码
2   def splitFfRegions(layers, shellThk, endLayerNum):  # 定义切片密实填充区分区函数
3       layers.insert(0, Layer(0))   为什么?            # 在0层切片前插入一个空切片
4       i, j = 0, 1                                     # 层序号，j > i，相邻层
5       while True:                                     # 第一重循环，遍历所有切片
6           if j > len(layers) - 1:                     # 第一重循环退出条件
7               break
8           while pickFfRegions(layers[i], layers[j], shellThk):  # 第二重循环，建立端面厚度
9               j = j + 1
10              if j > len(layers) - 1 or j - i > endLayerNum:  # 第二重循环到达endLayerNum
11                  break
12          i = i + 1 if j - i == 1 else j – 1           # 计算下一个需要遍历的i
13          j = i + 1                                    # 计算下一个需要遍历的j
14      del layers[0]                                    # 删除之前插入的空切片层
```

splitFfRegions 函数主要包含两重循环

- **第一种循环**的作用是从第0、1层切片开始，调用**pickFfRegions**函数从下往上逐渐遍历切片序列layers中的所有相邻切片对；一旦在某层切片上发现端面，则开启第二重循环。
- **第二重循环**的作用是根据指定的端面切片层数 **endLayerNum** 建立端面厚度，它有两个循环结束条件。其中第一个条件是在调用 **pickFfRegions** 函数时，如果发现 ci 和 cj 轮廓见不存在下端面，此时第二重循环提前结束；第二个条件是循环到达指定的端面切片层数，此时第二重循环正常结束。

- **输入参数**
- **layers**　　——为切片序列，该参数可对指定三维模型切片得到；
- **shellThk**　　——表示每层切片上的轮廓路径填充带宽，该参数将被输入pickFfRegions函数中；
- **endLayerNum**　　——表示密实填充的端面切片层数，即公式(11-5)中的变量n。

插入空切片原因：layers的**底层切片必须使用密实填充**；但由于底层切片下方已无切片，因此，无法用pickFfRegions函数为其提取密实填充区域。借助上述虚拟切片层，可以用**统一方法为模型底层切片生成密实填充区，而不用专门寻找新的方法**。

1. 端面识别与切片分区

- **1.2. 切片疏密填充区域分区**

splitFfRegions

可解决layers**下端面密实填充区域**提取的问题

那**上端面密实填充区**如何提取呢?

颠倒模型/切片，就能获取模型切片上端面的密实填充区

Python中列表取反序layers[::-1]

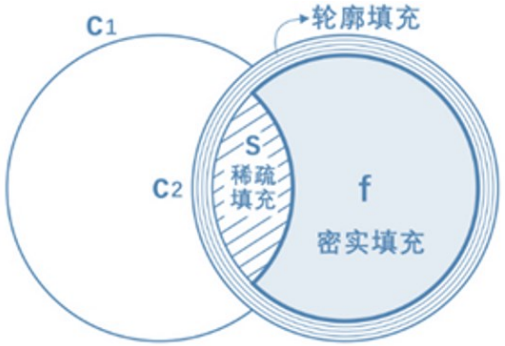
1. 端面识别与切片分区

- **1.2. 切片疏密填充区域分区：稀疏填充区寻找**

- 在splitFfRegions函数的基础上，下面根据式(11-3)继续给出针对**所有切片层**的**稀疏填充区域寻找函数**。仍将这个函数写在IdEndLayers模块中，函数名为**splitSfRegions**，具体实现如下：

IdEndLayers.py	定义所有切片层稀疏填充区域寻找函数	11-5

```
1   ……                                          # 导入相关库，忽略其他代码
2   def splitSfRegions(layers):                 # 定义切片稀疏填充区分区函数
3       ca = ClipperAdaptor()
4       for layer in layers:                    # 遍历所有切片
5           if len(layer.shellContours) > 0:    # 如果外壳内边界不为空
6               if len(layer.ffContours) == 0:  # 如果密实填充区为空
7                   layer.sfContours = layer.shellContours   # 则稀疏填充区轮廓为外壳内边界
8               else:                           # 否则根据式(11-3)计算稀疏填充区
9                   s = ca.clip(layer.shellContours, layer.ffContours, CT_DIFFERENCE, layer.z)
10                  layer.sfContours = s
```



$$s = c_{2oi} - f \qquad (11\text{-}3)$$

- **splitSfRegions** 函数的输入参数只有一个，即模型切片序列 layers。

- 在该函数中，对每层切片，首先检测外壳内边界 shellContours 是否为空，如果不为空，则进一步检测密实填充区轮廓 ffContours 是否为空。如果 ffContours 为空，则直接取 shellContours 为稀疏填充区轮廓 sfCountours，否则根据式(11-3)布尔运算得到 sfContours。

1. 端面识别与切片分区

- **1.2. 切片疏密填充区域分区：接口函数**

- 通过对**splitFfRegions**和**splitSfRegions**两个函数的进一步简单封装，在IdEndLayers模块中给出该模块的同名对外**接口函数idEndLayers**，该函数包含layers、shellThk和endLayerNum三个参数，具体实现如下：

IdEndLayers.py	定义模块接口函数，实现端面识别、切片疏密分区功能	11-6

```
1   ……                                          # 导入相关库，忽略其他代码
2   def idEndLayers(layers, shellThk, endLayerNum):   # 模型切片端面识别、疏密分区
3       splitFfRegions(layers, shellThk, endLayerNum)   # 生成上端面密实填充区
4       splitFfRegions(layers[::-1], shellThk, endLayerNum)   # 颠倒切片，生成下端面密实填充区
5       splitSfRegions(layers)                      # 生成所有切片稀疏填充区
```

输入参数

- **layers** ——为切片序列，该参数可对指定三维模型切片得到；
- **shellThk** ——表示每层切片上的轮廓路径填充带宽，该参数将被输入pickFfRegions函数中；
- **endLayerNum** ——表示密实填充的端面切片层数，即公式(11-5)中的变量n。

- 使用接口函数 **idEndLayers**，可识别模型切片中的上、下端面，同时对所有切片进行疏密分区。对任一切片轮廓，可能得到四种不同的分区结果：
  - ① **同时存在密实填充区和稀疏填充区；**
  - ② **存在稀疏填充区，但不存在密实填充区；**
  - ③ **存在密实填充区，但不存在稀疏填充区（如模型底面、顶面等）；**
  - ④ **同时不存在密实填充区和稀疏填充区。**

- 对第四种结果，如果出现，表明这层切片轮廓所围的**区域面积较小**，**仅外围的轮廓填充就能将整个区域覆盖**，此时当然不存在密实或稀疏填充区。

1. 端面识别与切片分区

- **1.3. 切片疏密分区测试**

- **IdEndLayers** 模块用于识别三维模型切片中的上、下端面，并为每层切片**生成稀疏填充区和密实填充区**，是三维打印填充路径生成的前续基础模块。
- 下面对 IdEndLayers 模块进行测试，检验它在**功能**上是否能够满足需求，同时查看它的**执行性能**。简单起见，将测试代码直接写在IdEndLayers模块中，具体如下：

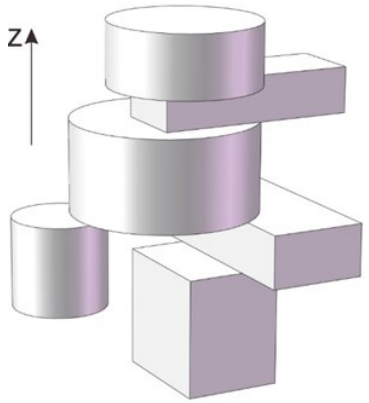
IdEndLayers.py	IdEndLayers模块测试	11-7

```
1   ……                                          # 导入相关库，忽略其他代码
2   if __name__ == '__main__':
3       src, stlModel = vtk.vtkSTLReader(), StlModel()
4       src.SetFileName("E:\\Models\\STL\\multiEnds.stl")  # 读取STL模型文件
5       stlModel.extractFromVtkStlReader(src)              # 提取VTK数据源
6       layerThk, shellThk, endThk = 0.5, 2.0, 3.0         # 设置切片、外壳、端面厚度参数
7       endLayerNum = int(endThk / layerThk) + 1           # 计算端面切片层数
8       layers = slice_topo(stlModel, layerThk)            # 对输入三维模型切片
9       idEndLayers(layers, shellThk, endLayerNum)         # 识别切片端面并生成疏密分区
10      va = VtkAdaptor()                                  # 显示切片分区轮廓
11      for layer in layers:
12          for poly in layer.contours:                    # 显示切片原始轮廓，黑色
13              va.drawPolyline(poly) .GetProperty()..SetColor(0, 0, 0)
14          for poly in layer.ffContours:                  # 显示密实填充区轮廓，红色
15              va.drawPolyline(poly) .GetProperty()..SetColor(1, 0, 0)
16          for poly in layer.sfContours:                  # 显示稀疏填充区轮廓，绿色
17              va.drawPolyline(poly) .GetProperty()..SetColor(0, 1, 0)
18      va.display()
```

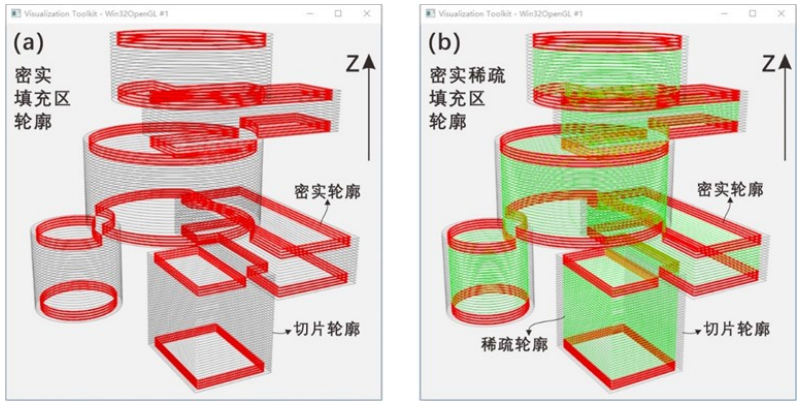
1. 端面识别与切片分区

- **1.3. 切片疏密分区测试：第1个测试模型：MultiEnds.stl —— 功能验证**
- **模型特点：**简单几何体，Z向高度120mm，由若干圆柱体、长方体拼叠而成，有**多个上、下端面。**
- **测试参数：**
  - 切片厚度（layerThk）为1.0mm
  - 外壳厚度（shellThk）为2.0mm
  - 端面厚度（endThk）为3.0mm

根据公式(11-5)，可得端面层数（endLayerNum）为 4。



包含多个端面的简单三维模型



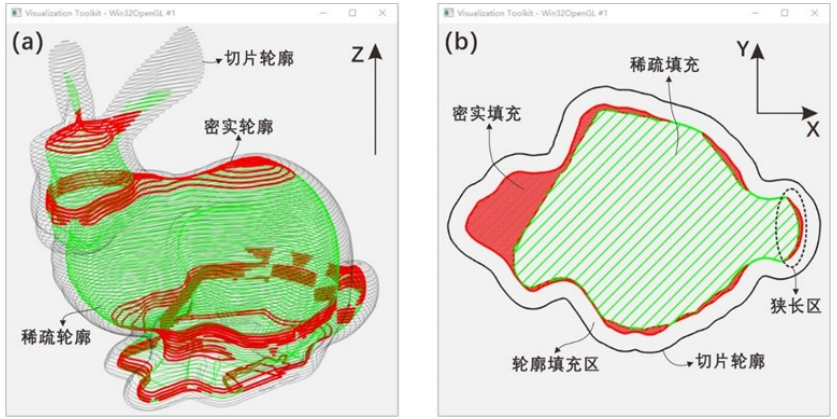
multiEnds模型端面识别与疏密分区计算结果

- 从上图可以看出，idEndLayers函数对模型上各个**上、下端面进行了正确无误地识别**，并且，在端面附近，生成了一系列密实填充区轮廓（粗线），而在其他切片上，生成了一系列稀疏填充区轮廓（细线）。
- 值得一提的是，在简单几何体相交处（如圆柱与长方体相交处），对相交重叠区域，得到的是稀疏填充区轮廓；而对不重叠区域，得到的是密实填充区轮廓。这些结果都是符合预期设想的，而且是正确的。
- 通过以上测试，可以得出，**IdEndLayers 模块在功能上是能够满足要求的。**

1. 端面识别与切片分区

- **1.3. 切片疏密分区测试：第2个测试模型：bunny.stl　——性能（效率）验证**
- **模型特点：**约27万个三角面片，Z向高度约为49.5mm。
- **测试参数：**
  - 切片厚度（layerThk）为0.5mm
  - 外壳厚度（shellThk）为2.0mm
  - 端面厚度（endThk）为2.0mm

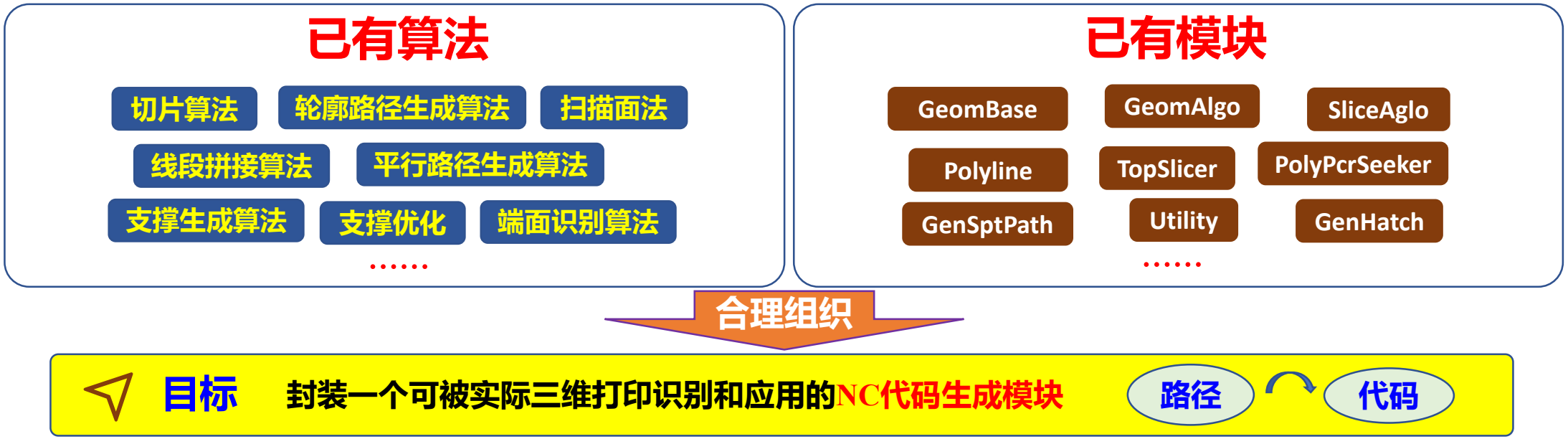
根据公式(11-5)，可得端面层数（endLayerNum）为 5。



bunny模型端面识别与疏密分区计算结果

- 在测试平台（CPU为intel Core i7-8700 @ 3.20GH）上，**slice_topo 函数的运行时间约为 9 秒**，该函数生成了bunny模型的98层切片，对这些切片，**idEndLayers 函数的运行时间约为 2秒**。对bunny疏密分区结果如图a。
- 图b给出了bunny模型其中一层切片上的疏密分区结果。根据该图，这层切片并不是具有典型意义的端面切片。它在右侧和上、下层切片轮廓之间存在一定的偏差，**虽然偏差都很小**，但已经满足式(11-4)中 **e 不为空的条件**，这就导致在该层切片轮廓边缘附近，存在一系列**狭长的密实填充区域**。

2. 打印代码生成



说明：

- 在计算机辅助制造或路径规划中，**“路径”**是相对于工件模型和工艺参数的**几何概念**，通常指计算机内存中的生成或存储的**曲线（或多段线）**数组；
- 而**“NC代码”具有设备属**性，它是相对于具体数控设备而言的，如数控机床、三维打印机等，数控设备通过读取、解析NC代码控制各进给轴运动、I/O执行相应操作。
- **从曲线路径到NC代码的转化过程一般叫做后置处理**。后置处理需要考虑目标设备特性，对相同路径，针对不同目标设备可能生成不同的NC代码。

2. 打印代码生成

• 2.1. 工艺参数汇总

• **三维打印路径规划流程包括**：**三维模型切片→端面识别（含切片疏密分区）→轮廓填充路径生成→平行填充路径生成→支撑路径生成。与之对应函数包括**：**slice_topo（或slice_combine）、idEndlayer、genCpPath、genDpPath、genSptPath。**因此，只要提取、汇总这些函数的各个输入参数，就能大致构建三维打印路径规划的工艺参数表。

序号	参数名	数据类型	相关函数	描述
1	**stlModel**	StlModel	slice_topo、slice_combine	STL模型
2	**layerThk**	float		切片厚度
3	**shellThk**	float	idEndlayer	外壳厚度（轮廓路径填充宽度）
4	**endThk**	float		端面厚度，用于计算端面切片层数
5	**sfRate**	float	genDpPath	平行路径稀疏填充率
6	**fillAngle**	float		平行路径填充角度
7	**sptOn**	bool	genSptPath	是否使用支撑
8	**sptSfRate**	float		支撑部分平行路径填充率
9	**sptGridSize**	float		模型投影底面网格尺寸
10	**sptCrAngle**	float		危险面片临界角度
11	**sptFillType**	枚举		支撑填充类型，交叉或同向
12	**sptFillAngle**	float		支撑部分平行路径填充角度
13	**sptXyGap**	float		支撑到路径横向间距
14	**nozzleSize**	float	genCpPath、genDpPath、genSptPath	FDM打印机喷头尺寸（直径）
15	**filamentSize**	float		FDM打印丝材尺寸（直径）
16	**g0Speed**	int		G0速度，即空走速度，不出丝
17	**g1Speed**	int		G1速度，即打印速度，出丝
18	**startCode**	字符串		起始代码，和具体打印机相关
19	**endCode**	字符串		结束代码，和具体打印机相关

2. 打印代码生成

• 2.1. 工艺参数汇总：参数解释

- 在表中，有些参数可能和上述函数**不存在直接关系**，但它们是**间接相关**的。
  - 如**sfRate**，该参数表示工件内部稀疏填充区域平行路径的填充率，取值为大于0且小于等于1，它和喷头尺寸（nozzleSize）一起可用于计算稀疏平行填充间距（sfInvl），计算公式如下：

$$sfInvl = nozzleSize / sfRate \qquad (11\text{-}6)$$

  - 得到的sfInvl可作为参数输入genDpPath函数计算稀疏平行填充路径。同理**sptSfRate**表示支撑部分平行填充路径填充率，可用于计算支撑部分平行填充间距。相对于直接给出平行路径填充间距的具体数值，**用填充率显得更直观，且对用户更友好**。

- 在表中，根据三维打印实际需求，额外添加了一些参数。
  - **sptOn**：这是一个开关量，表示是否需要生成支撑路径。
  - **nozzleSize**、**filamentSize**：分别表示FDM打印机喷头直径尺寸和丝材直径尺寸，其中nozzleSize可用于计算路径间距（前面已提及），而nozzleSize和filamentSize一起可用于计算挤丝电机的转动量。
  - **g0Speed**、**g1Speed**：分别表示快速进给（不出丝）和打印进给（出丝）速度，单位一般为mm/min（视具体打印机而定），它们和路径曲线形状无关，但和NC代码中各轴进给速度有关。
  - **startCode**、**endCode**：用于指定NC代码中的开始代码和结束模板，在开始代码中，可以写入各轴回零、坐标系统选择、打印温度设置、风扇设置等代码；而在结束代码中，可以写入程序结束、打印机重置等代码。

2. 打印代码生成

• 2.1. 工艺参数汇总：代码实现

- 为更好地组织、管理表中所有工艺参数，专门在 **GenNcCode** 模块中创建了一个 **PrintParams** 类，该模块后续将用于存放和NC代码生成相关的程序代码。**PrintParams** 类的具体实现如下：

GenNcCode.py	NC代码生成模块，定义打印工艺参数类	11-8
1	……	# 导入相关库
2	**class PrintParams:**	# 定义打印工艺参数类
3	**def __init__(self, stlModel):**	# 类的初始化函数
4	self.stlModel = stlModel	# STL模型
5	self.layerThk, self.shellThk, self.endThk = 0.2, 2.0, 2.0	# 切片、外壳、端面厚度
6	self.sfRate, self.fillAngle = 0.2, 0.0	# 工件内稀疏填充率、填充角度
7	self.sptOn = False	# 是否使用支撑，默认为否
8	self.sptCrAngle = degToRad(60.0)	# 危险面片临界角度
9	self.sptGridSize = 2.0	# 支撑网格尺寸
10	self.sptSfRate, self.sptFillAngle = 0.15, 0.0	# 支撑填充率，填充角度
11	self.sptFillType = SptFillType.cross	# 支撑填充类型，默认交叉填充
12	self.sptXyGap = 1.0	# 支撑到模型横向间距
13	self.g0Speed, self.g1Speed = 5000, 1000	# G0、G1速度
14	self.startCode = "; Start code...\n"	# 起始代码
15	self.endCode = "; End code...\n"	# 结束代码
16	self.nozzleSize, self.filamentSize = 0.5, 1.75	# 喷头、丝材直径尺寸

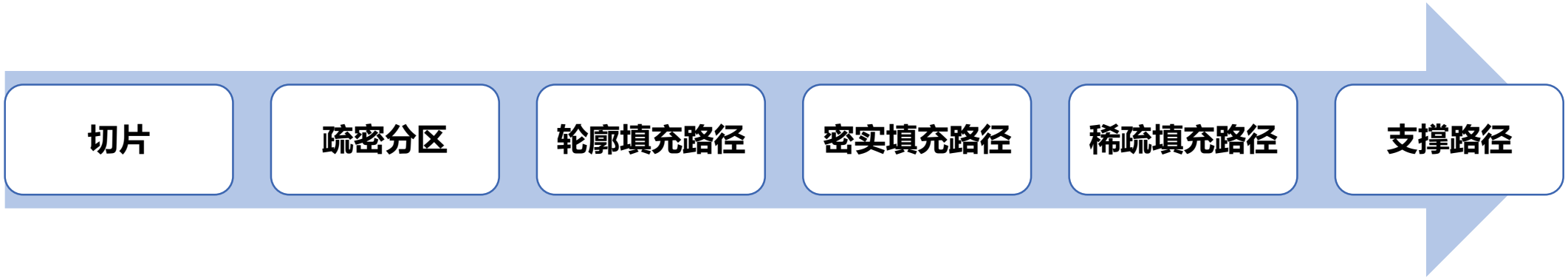
- **PrintParams** 类仅包含1个输入参数，即stlModel。类中大部分属性都给了默认值，如默认layerThk、shellThk、endThk分别为0.2mm、2.0mm、3.0mm，默认sfRate为0.2，即工件内部稀疏填充率为20%，默认平行路径填充角度（包括工件内部和支撑部分）为0度，默认sptOn为False，即不生成支撑，默认危险面片临界角为60度等。
- 对**startCode**和**endCode**两个属性，这里并未对它们进行赋值（**G代码中，英文分号表示注释**），对这两个属性的赋值需要参考具体使用的打印机说明书。同理，对**nozzleSize**和**filamentSize**两个属性，尽管这里给出了它们的常规取值，但在具体使用时也要结合实际使用的喷头和丝材等情况。
- **注意：**根据实际需求，**可添加新的工艺参数**，如对模型进行轻量化**抽壳操作**所需的壁厚参数、对**切片轮廓进行简化**（减少轮廓顶点数）所需的误差参数等等。

2. 打印代码生成

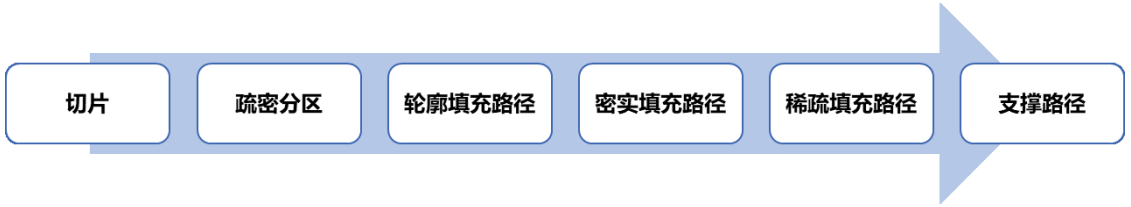
- **2.2. 切片路径生成**

- 在PrintParams类基础上，下面将对包括模型切片、端面识别、路径生成、支撑生成等过程涉及的函数进行**组织**，封装一个用于三维打印路径生成的综合函数**genAllPaths**，该函数核心步骤如下：

  - **步骤1.** 对三维模型进行**切片**（slice_topo或slice_combine）；
  - **步骤2.** 识别端面切片，并对切片轮廓进行**疏密分区**（idEndLayers）；
  - **步骤3.** 对各切片生成**轮廓填充路径**（genCpPath）；
  - **步骤4.** 对各切片生成**密实填充路径**（genDpPath）；
  - **步骤5.** 对各切片生成**稀疏填充路径**（genDpPath）；
  - **步骤6.** 如有必要，生成**支撑路径**（genSptPath）。



2. 打印代码生成



• 2.2. 切片路径生成

GenNcCode.py	封装模型切片、端面识别、路径生成、支撑生成等函数，生成打印路径	11-9

```
1   ……                                                          # 导入相关库，忽略其他代码
2   def genAllPaths(pp : PrintParams):                          # 定义路径生成函数
3       sfInvl = pp.nozzleSize / pp.sfRate                      # 计算工件内稀疏平行填充间距
4       sptSfInvl = pp.nozzleSize / pp.sptSfRate                # 计算支撑部分平行填充间距
5       layers = slice_topo(pp.stlModel, pp.layerThk)           # 1.对模型切片，得到切片序列
6       idEndLayers(layers, pp.shellThk, int(pp.endThk / pp.layerThk) + 1)   # 2.端面识别，疏密分区
7       for i, layer in enumerate(layers):                      # 遍历切片生成路径
8           layer.cpPaths, layer.ffPaths, layer.sfPaths = [], [], []        # 定义轮廓、密实、稀疏填充路径
9           layer.cpPaths = genCpPath(layer.contours, pp.nozzleSize, pp.shellThk)   # 3.生成轮廓路径
10          delta = 0 if i % 2 == 0 else math.pi / 2            # 平行路径上、下层交叉填充
11          if len(layer.ffContours) > 0:                       # 4.生成密实填充路径
12              layer.ffPaths = genDpPath(layer.ffContours, pp.nozzleSize, pp.fillAngle + delta)
13          if len(layer.sfContours) > 0:                       # 5.生成稀疏填充路径
14              layer.sfPaths = genDpPath(layer.sfContours, sfInvl, pp.fillAngle + delta)
15      if pp.sptOn:                                            # 6.生成支撑路径
16          genSptPath(pp.stlModel, layers, sptSfInvl, pp.sptGridSize, pp.sptCrAngle, \
17                     pp.sptFillType, pp.sptFillAngle, pp.sptXyGap)
18      return layers                                           # 返回切片序列
```

- 针对每层切片，总共生成了**5种打印路径**，包括：工件部分的轮廓路径 **cpPaths**、密实填充路径 **ffPaths**、稀疏填充路径 **sfPaths**，以及支撑部分的轮廓路径 **sptCpPaths** 和填充路径 **sptDpPaths** 。
- 这5种路径属性都通过动态方式添加至 **Layer** 类。

2. 打印代码生成

- **2.3. 路径代码转换：G代码**

- 对常见的 FDM 三维打印机来说，它能接收的 NC 代码一般就是指 **G 代码**。G 代码是数控加工领域最为广泛使用的编程语言，是人类和数控设备沟通的桥梁之一。
- 标准的 G 代码一般包含 **G（准备）**和 **M（辅助）**两种功能字，分别由地址符 G、M 和其后的数字组成。
  - **G 功能**：数控插补、刀具补偿、坐标选择、循环控制等功能；
  - **M 功能**：主轴旋转方向、冷却液开关、程序启停、刀具更换等功能。
- 对FDM三维打印机来说，它能接收的 G 代码是标准 G 代码的一个**子集**（或对子集的修改）。这里只介绍和打印路径本身相关的几个指令，包括G0、G1、G90、G91，它们的具体功能如下：

序号	指令	功能	示例
1	**G0**	快速定位，不出丝	G0 F2000 X10.1 Y 20.2 Z 30.3
2	**G1**	直线插补，打印出丝	G1 F1000 X20.1 Y 30.2 Z 40.3 E50.1
3	**G90**	指定绝对坐标编程	G90
4	**G91**	指定增量坐标编程	G91

2. 打印代码生成

- **2.3. 路径代码转换：G0 和 G1**
- 在NC代码中，一般使用G0和G1两个指令来**跟踪、描绘打印路径**。
- **G0为快速定位指令**，可控制打印喷头从一个点快速移动到另一点，期间喷头**不出丝**，通常被用于两段打印路径之间的衔接。
  - 在实际使用中，需要在 G0 指令后面添加速度指令（F字段）和终点坐标指令（X、Y、Z字段），移动的起点就是当前所处点。
- **G1为直线插补指令**，可控制打印喷头从一个点沿直线轨迹移动到另一点，期间喷头**出丝**。
  - 在实际使用中，需要在 G1 后面添加 F、X、Y、Z 的具体数值；此外，为了控制喷头出丝，还需要为其添加**进丝指令（E字段，表示进丝长度）**。

说明：G0 和 G1 都是**模态指令**（续效指令），即在G0或G1引导的代码中，如果当前行代码中的某个字段相对于上一行代码相同字段的数值未发生改变，那么可以在当前行代码中省略这个字段，如对以下两行代码：

```
1    G1 F1000 X10 Y20 Z30 E40    ; (10, 20, 30)
2    G1 F1000 X30 Y40 Z30 E50    ; (30, 40, 30)
```

简写

```
1    G1 F1000 X10 Y20 Z30 E40    ; (10, 20, 30)
2    G1 X30 Y40 E50              ; (30, 40, 30)
```

甚至可以省略第二行中的“G1”。注意，在G代码中，**使用英文分号“;”进行代码注释**。

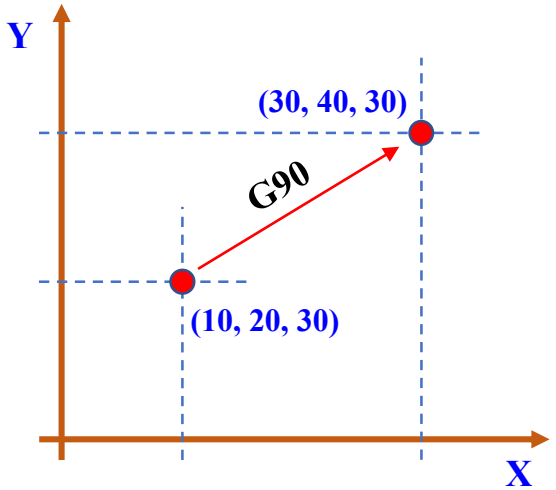
2. 打印代码生成

- **2.3. 路径代码转换：G90 和 G91**

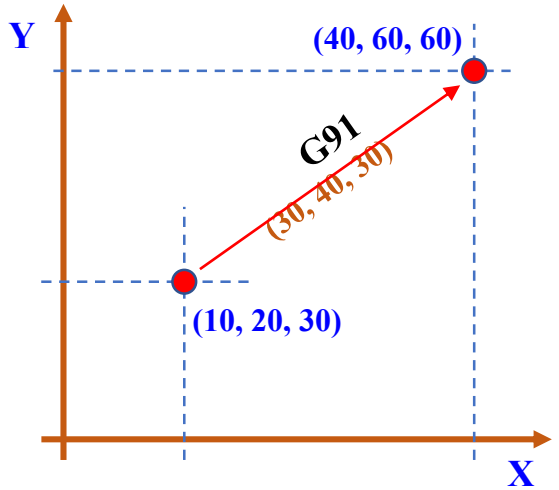
- 在NC代码中，**G90** 表示**绝对编程**，**G91** 表示**增量（相对）编程**。
- 在FDM三维打印中，简单起见，**默认使用绝对坐标编程**，即使用G91指令。

1	G1 F1000 X10 Y20 Z30 E40 ; (10, 20, 30)
2	G1 F1000 X30 Y40 Z30 E50 ; (30, 40, 30)

- 如果采用绝对编程，那么**(10, 20, 30)**和**(30, 40, 30)**就是执行两段代码后的终点坐标。
- 如果采用增量编程，那么终点坐标分别为**(10, 20, 30)**和**(40, 60, 60)**（假设(10, 20, 30)本身等于绝对坐标）。



G90绝对编程



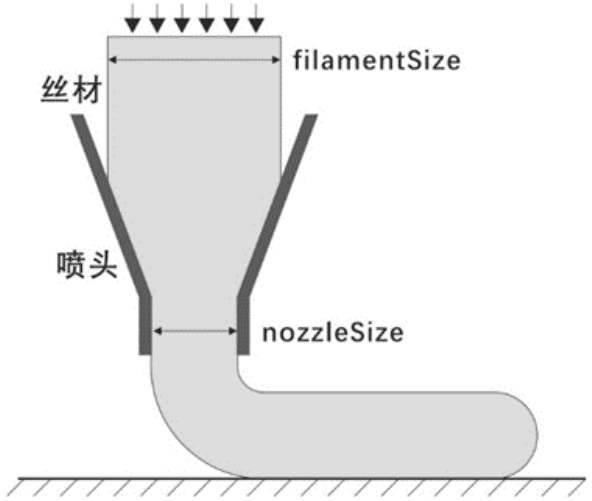
G91相对编程

2. 打印代码生成

- **2.3. 路径代码转换：进丝指令 E**

- G代码中的 **X、Y、Z 坐标数值**是直接可以从**路径曲线顶点**中提取得到的，**那进丝指令 E 的数值又该如何得到呢?**
- 在FDM三维打印中，打印丝材在打印喷头的加热和进丝电机的挤压作用下，从喷头挤出。通常情况下，**喷头直径小于丝材直径**，即从喷头**前进去的丝材长度**要**小于**从**喷头挤出来的丝材长度**，它们之间的具体关系如下：

$$\frac{L_{in}}{L_{out}} = \left(\frac{nozzleSize}{filamentSize}\right)^2, \qquad (11\text{-}7)$$



FDM三维打印喷头进出丝示意

- ✓ 式中 L_{in} 为进丝长度，L_{out} 为出丝长度。
- ✓ **在知道出丝长度的情况下，根据式(11-7)，可计算相应的进丝长度。**
- ✓ 在FDM三维打印中，喷头**喷出的丝材长度**等于由 **G1 指定和累加的路径总长度**，因此，在已知打印路径的前提下，可以计算得到相应的出丝长度的。
- ✓ 综上所述，对路径长度进行累加，同时结合式(11-7)对长度进行转换，即可对 G 代码中的进丝量 E 赋值。

2. 打印代码生成

- **2.3. 路径代码转换：pathToCode**

- 下面给出将一条路径曲线“翻译”成G代码的函数，函数名为 **pathToCode**，它的具体实现如下：

GenNcCode.py	将一条路径曲线（多段线）转化为NC代码	11-10

```
1   ……                                          # 导入相关库，忽略其他代码
2   def pathToCode(path, pp, e, nf2):           # 定义路径到NC代码转化函数
3       code = ""                               # 定义文本变量code
4       for i, p in enumerate(path.points):     # 遍历多段线路径中所有顶点
5           if i == 0:                          # 使用G0快速跳至路径起点
6               code += "G0 F%d X%.3f Y%.3f Z%.3f\n" % (pp.g0Speed, p.x, p.y, p.z)
7           else:                               # 使用G1打印路径上的其他点
8               e += p.distance(path.points[i-1]) * nf2   # 累加得到e值，即进丝量
9               code += "G1 F%d X%.3f Y%.3f E%.3f\n" % (pp.g1Speed, p.x, p.y, e)
10      return code                             # 返回代码文本
```

- **pathToCode**函数的**核心功能**就是将路径上各顶点坐标按G代码标准格式化为字符串。
- **pathToCode**函数包含4个输入参数，其中：
  - **path** **是Polyline类型的多段线**
  - **pp** **为PrintParams类型的打印参数**
  - **e** **为累计进丝长度**
  - **nf2** **为喷头尺寸（nozzleSize）和丝材尺寸（filamentSize）比值的平方（参见式(11-7)）**
- 考虑到genAllPaths函数可能生成很多条路径，在将这些路径转化为NC代码时，可能需要多次调用pathToCode函数，此时**参数e可以贯穿这些调用，得到累加进丝长度**。
- **参数nf2**，将其作为输入参数而不是在pathToCode函数中直接计算纯粹是出于**降低计算量的考虑**。

2. 打印代码生成

- **2.3. 路径代码转换： pathToCode**
- 在 **pathToCode** 函数中，对**路径起点使用了G0指令**，让打印喷头以速度g0Speed快速移动至该点，且移动过程中喷头不出丝。
  - **同一切片**：对FDM三维打印，在同一层切片上，当喷头从一点平移至另一点时，考虑到平移过程中无障碍物，因此无需设置安全平移高度；
  - **不同切片**：当喷头完成一层打印，向上移动一层时，由于喷头Z向高度提升，也不会碰到障碍物，因此也无需设置安全移动高度。
- 综上两种情况，在路径起点，只要让喷头从上一条路径终点直接以G0移动至当前路径起点即可。
- 对**路径上的其他点使用了G1指令**，让喷头以速度g1Speed从前一点沿直线移动至相应点，且移动过程中喷头出丝，出丝增量根据路径上当前点到前一点的距离及nf2计算得到。
- 函数**pathToCode**的返回为指定路径的**NC代码文本**。使用pathToCode函数，可以获取由genAllPaths函数生成的所有路径的所有NC代码。只要将所有NC代码进行累加，就能生成最终NC代码。

2. 打印代码生成

- **2.3. 路径代码转换：postProcess**

- 最后在 GenNcCode 模块中给出打印路径的后处理函数，函数名为 **postProcess**，具体实现如下：

GenNcCode.py	NC代码生成模块，定义打印参数类	11-11

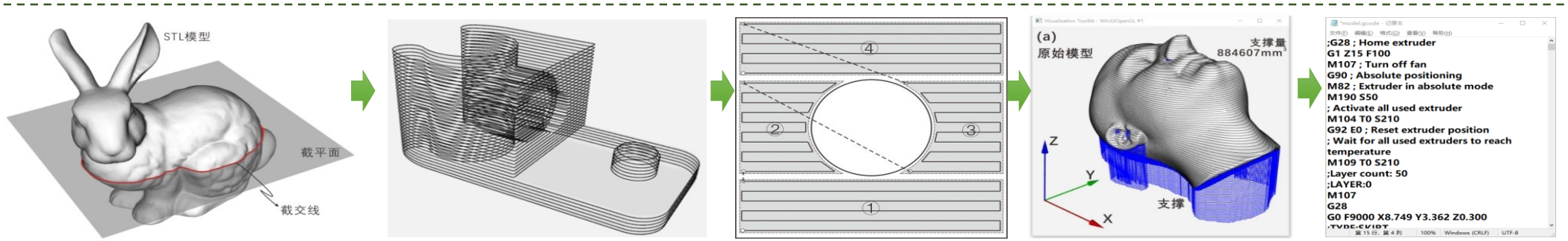
```
1   def postProceess(layers, pp):                          # 三维打印路径后处理函数
2       code, e = pp.startCode, 0                          # 定义代码文本code和进丝量e
3       nf2 = (pp.nozzleSize / pp.filamentSize)**2         # 计算喷头和丝材尺寸比值平方
4       for i, layer in enumerate(layers):                 # 遍历每层切片中的5种路径
5           code += "; Layer %d\n" % I                     # 在代码中标识当前层号
6           for path in layer.sptCpPaths: code += pathToCode(path, pp, e, nf2)     # 支撑轮廓路径代码
7           for path in layer.sptDpPaths: code += pathToCode(path, pp, e, nf2)     # 支撑填充路径代码
8           for path in layer.cpPaths: code += pathToCode(path, pp, e, nf2)        # 轮廓路径代码
9           for path in layer.ffPaths: code += pathToCode(path, pp, e, nf2)        # 密实填充路径代码
10          for path in layer.sfPaths: code += pathToCode(path, pp, e, nf2)        # 稀疏填充路径代码
11      return code + pp.endCode                           # 返回NC代码
12  def genNcCode(pp : PrintParams):                       # 定义NC代码生成函数
13      layers = genAllPaths(pp)                           # 生成打印路径
14      return postProcess(layers, pp)                     # 打印路径后处理，生成NC代码
```

- **postProcess**函数的**输入**为切片序列layers和打印参数pp。注意：这里的**layers中已包含打印路径**。
  - 每层切片上至多有**5种路径**，即工件部分的轮廓路径**cpPaths**、密实填充路径**ffPaths**、稀疏填充路径**sfPaths**，以及支撑部分的轮廓路径**sptCpPaths**和填充路径**sptDpPaths**，针对这些路径，分别调用pathToCode函数生成了相应的代码。
  - 在每层切片路径打印顺序的安排上，采用了“**先支撑后工件、由外至内**”的策略，即先打印支撑部分，后打印工件部分，对每个部分，先打印外围轮廓路径，后打印内部填充路径。

- 在**postProcess**函数中，对累加得到的NC代码，在其开头添加了来自打印参数pp的起始代码startCode，并在结尾添加了结束代码endCode。在实际应用中，如果有需求，只需在修改pp中的相应参数即可。
  - 为了标记NC代码中的切片层信息，在每层开头以注释的形式添加了**层序号**。
  - 同理，也可以在每层切片的每种路径前添加相应的**注释文本**，方便后续查看。

2. 打印代码生成

- **2.3. 路径代码转换：genNcCode**
- 在GenNcCode模块最后，定义了一个 GenNcCode 模块的同名接口函数 genNcCode。
  - genNcCode函数的输入参数为打印参数pp，输出为NC代码文本。
  - genNcCode函数非常简洁，即先调用 genAllPaths 生成打印路径，在调用 postProcess 对路径进行后处理，输出NC代码。
  - 需要说明的是，在 genNcCode 函数中，并未将模型优化摆放问题纳入考虑。
- genNcCode 函数的重要价值不仅仅是相对GenNcCode模块来说的，它也是本课程所有理论、算法，所有模块、类、函数，所有尝试、实践、测试的最终结果与出口！



三维模型　　切片生成　　填充路径生成　　支撑生成　　NC代码

本章小结

- 这一章主要完成了三维打印路径规划中的最后两个任务：
  - **一是对模型切片上的端面识别问题展开了研究。**
    - 为了封闭三维打印后得到的工件，有必要对模型上的端面进行识别，并对端面区域进行密实填充；同时为了节省打印材料和时间，对除端面外的其他区域，可以进行稀疏填充。
    - 为此，我们提出了一种基于平面轮廓布尔运算和偏置运算的端面切片识别方法，并在此基础上，将各层切片分成密实填充区、稀疏填充区和轮廓填充区3种区域。
    - 这部分工作是对三维打印路径规划的进一步补充和完善，封装了一个**IdEndLayers**模块。
  - **二是封装了一个三维打印NC代码生成模块GenNcCode。**
    - 通过对已得到的模型切片、端面识别、路径生成、支撑生成等相关函数输入参数的梳理，建立了三维打印工艺参数表。
    - 在此基础上，通过对相关函数的合理组织，构建了一个综合打印路径生成函数。
    - 结合当下常用的FDM三维打印机及其NC代码格式，封装了一个后处理函数，可将路径转化为三维打印机能够识别的NC代码。
    - 使用GenNcCode模块中的genNcCode函数，针对指定三维模型和工艺参数，可以生成相应的NC代码。





思考题

1. 在三维打印中，为什么需要对模型端面上的切片进行识别？
2. 如何判断某一层切片上是否存在上端面或下端面？
3. 探究模型上端面和支撑的关系。能否利用端面为模型添加支撑？
4. 在代码11-4中，简述在首层切片之前插入一层空切片的作用？
5. 简述在FDM三维打印的工艺参数中，startCode和endCode的作用。
6. 说明代码11-10存在的问题，如何改进？

本章小结

- 这一章主要完成了三维打印路径规划中的最后两个任务：
  - **一是对模型切片上的端面识别问题展开了研究。**
    - 为了封闭三维打印后得到的工件，有必要对模型上的端面进行识别，并对端面区域进行密实填充；同时为了节省打印材料和时间，对除端面外的其他区域，可以进行稀疏填充。
    - 为此，我们提出了一种基于平面轮廓布尔运算和偏置运算的端面切片识别方法，并在此基础上，将各层切片分成密实填充区、稀疏填充区和轮廓填充区3种区域。
    - 这部分工作是对三维打印路径规划的进一步补充和完善，封装了一个**IdEndLayers**模块。
  - **二是封装了一个三维打印NC代码生成模块GenNcCode。**
    - 通过对已得到的模型切片、端面识别、路径生成、支撑生成等相关函数输入参数的梳理，建立了三维打印工艺参数表。
    - 在此基础上，通过对相关函数的合理组织，构建了一个综合打印路径生成函数。
    - 结合当下常用的FDM三维打印机及其NC代码格式，封装了一个后处理函数，可将路径转化为三维打印机能够识别的NC代码。
    - 使用GenNcCode模块中的genNcCode函数，针对指定三维模型和工艺参数，可以生成相应的NC代码。

